



TEST DOCUMENT (UNIT & INTEGRATION)

Document 08 of 09 · B2B Data Engineering Platform



Test strategy, environment, criteria and results for unit, integration and KPI validation testing.
Detailed cases are in the Test Case Register.

Document ID	B2B-DEP-08
Version	1.2
Status	Final
Date	05/10/2026
Prepared by	Syed Kazim Raza
Reviewed by	[Team Lead]

05/10/2026

DATAROPES.AI

Data Engineering Internship · B2B Data Engineering Platform

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	-	Syed Kazim Raza	Template created
1.0	29/09/2026	Syed Kazim Raza	Test results of the final run (29/09/2026)
1.1	30/09/2026	Syed Kazim Raza	Re-tested on the rebuilt data: 186 tests passed; quarantine tests; defect log
1.2	05/10/2026	Syed Kazim Raza	Formatting to documentation standard: table column widths, numbered figure and table captions with lists, figure descriptions, complete acronym list

Approvals

Role	Name	Signature	Date
Prepared by	Syed Kazim Raza		
Reviewed by	[Team Lead]		
Approved by	[Manager]		

Related Documents

Ref	Document	Location
R1	Business Requirements Document (BRD)	Provided by DataRopes
R2	Project Document (technical reference)	docs/deliverables/project_document.md
R3	Project Plan	docs/deliverables/project_plan.md
R4	README / Deployment Guide	README.md

Contents

Document Control	2
Contents	3
List of Figures.....	4
List of Tables.....	5
1. Introduction.....	6
1.1 Purpose.....	6
1.2 Scope	6
1.3 Intended Audience	6
1.4 Definitions & Acronyms.....	6
2. Test Strategy.....	7
3. Test Environment	7
4. Test Data.....	7
5. Entry & Exit Criteria	8
6. Unit Test Summary	8
7. Integration Test Summary	9
8. KPI Validation.....	10
9. Defect Log.....	11
10. Test Results Summary	12
11. Test Evidence.....	12
12. Sign-off.....	21

List of Figures

Figure 1: Pipeline run log — all stages completed	12
Figure 2: Unit test query returning zero rows — intermediate	13
Figure 3: Unit test query returning zero rows — warehouse.....	14
Figure 4: SCD Type 2 — companies with more than one version	15
Figure 5: Idempotency re-run — row counts unchanged	16
Figure 6: Layer reconciliation — every CSV row is loaded or rejected.....	17
Figure 7: Rejected rows by table and reason	18
Figure 8: Integration test summary — failed_rows is 0 for every test.....	19
Figure 9: Terminal after an incremental run — staging checked and cleared	20

List of Tables

Table 1: Intended Audience..... 6

Table 2: Definitions & Acronyms 6

Table 3: Test Strategy 7

Table 4: Test Environment..... 7

Table 5: Entry & Exit Criteria 8

Table 6: Unit Test Summary 8

Table 7: Integration Test Summary 9

Table 8: KPI Validation..... 10

Table 9: Defect Log 11

Table 10: Test Results Summary..... 12

Table 11: Sign-off..... 21

1. Introduction

1.1 Purpose

This document defines how the platform is tested and records the results.

1.2 Scope

In scope:

- Unit tests per table/object, integration tests across layers, end-to-end pipeline runs, KPI validation

Out of scope:

- Performance and load testing at production scale
- Security penetration testing

1.3 Intended Audience

Table 1: Intended Audience

Audience	How they use this document
QA / Data Engineers	Execute tests and record results
Team Lead	Review coverage and sign off

1.4 Definitions & Acronyms

Table 2: Definitions & Acronyms

Term	Definition
B2B	Business-to-Business: companies selling to other companies.
BI	Business Intelligence: reporting and dashboards, here Power BI.
BRD	Business Requirements Document: here the Project Requirements Document provided by DataRopes.
CDC	Change Data Capture: extracting only the rows that changed since the last run (here: watermark on updated_at).
CI/CD	Continuous Integration / Continuous Delivery: automated build, test and deployment (out of scope for this project).
CSV	Comma-Separated Values: the text-file format of the marketing leads and web logs feeds.
DEP	Data Engineering Platform: prefix of the document IDs (B2B-DEP-01 ...).
ID	Identifier.
Integration test	A check across layers or tables (e.g. source rows = intermediate + rejected = warehouse).
IT	Integration Test: IT-01 ... IT-10 in the Test Document.
KPI validation	Comparing a KPI view's result with an independent calculation from the warehouse tables.
MoM	Month-over-Month: change compared with the previous month.
Read-only run	Tests are executed in a READ ONLY transaction, so testing never changes data.
SCD	Slowly Changing Dimension: SCD1 overwrites changes, SCD2 keeps every version with validity dates.
SQL	Structured Query Language.
Test register	08b_Test_Case_Register.xlsx: one row per test case with expected and actual result.
UAT	User Acceptance Testing: business users confirm the solution meets their needs.
Unit test	A SQL query that checks one rule on one table (e.g. no duplicate order_id). It passes when it returns 0 rows.

2. Test Strategy

Table 3: Test Strategy

Test Level	Objective	What Is Tested	Technique	Owner
Unit	Each table meets its data rules	Uniqueness, completeness, allowed values, ranges, referential integrity, SCD2 rules	SQL test queries per table (0 rows = pass)	Data Engineer
Integration	Data flows correctly between layers	Row reconciliation source -> intermediate (+ rejected) -> warehouse, watermarks, SCD2 history, lead-order link	Read-only reconciliation queries	Data Engineer
System / End-to-End	The whole pipeline runs end to end	initial and incremental runs of run_pipeline.py	Execute the pipeline, check run log and notifications	Data Engineer
KPI Validation	KPIs are correct	All 15 mart views	Compare view totals with independent SQL on the facts	Data Engineer
User Acceptance (UAT)	Business users accept the dashboard	Power BI dashboard, 7 pages	Demonstration and review	Team Lead

3. Test Environment

Table 4: Test Environment

Databases	SQL Server 2022 Express (b2b_source_db); PostgreSQL 18.4 (b2b_warehouse_db)
Data volume	470,263 order lines, 156,678 orders, 148,358 leads, 296,703 web requests (+ 6,775 rejected CSV rows)
Tools	pgAdmin 4 / psql for SQL tests; Python (SQLAlchemy) to execute all test files in one run; Power BI Desktop
Test data source	Synthetic data from the project generators: 2-year backfill plus daily incremental runs
Execution method	Manual end-to-end runs followed by the SQL test files (requirements: no pytest, no CI/CD)

4. Test Data

All tests use the synthetic data created by the project's generators, so no production data is needed:

- **Deterministic seeding:** RANDOM_SEED = 40 and metadata.Generation_Seeds make the reference data repeatable.
- **History:** a 2-year backfill (2,500 companies, 94,755 customers, 156,678 orders) with seasonal and weekday patterns.
- **Change cases (CDC):** each incremental run updates companies (SCD2 test cases), customers, order statuses and lead stages, and deactivates products (soft deletes).
- **Edge cases:** cancelled orders (7,613, 4.9 %), bot traffic, anonymous web requests, orders from leads and without a lead, and 1.5% broken CSV rows.

5. Entry & Exit Criteria

Table 5: Entry & Exit Criteria

Criteria	Entry	Exit
Unit	Pipeline run completed; intermediate and warehouse loaded	All unit tests return 0 rows (or matching counts)
Integration	Unit tests passed	All layers reconcile; watermarks and SCD2 history correct
KPI validation	Warehouse loaded and marts views created	Every KPI total equals its independent calculation

6. Unit Test Summary

Table 6: Unit Test Summary

Test File	Object Under Test	# Cases	Checks Covered	Result
Test_categories.sql	intermediate.categories	5	Completeness, Uniqueness	5 / 5 passed
Test_customers.sql	intermediate.customers	6	Business rule, Completeness, Consistency, Domain / allowed values	6 / 6 passed
Test_order_items.sql	intermediate.order_items	9	Business rule, Completeness, Consistency, Reconciliation	9 / 9 passed
Test_orders.sql	intermediate.orders	11	Business rule, Completeness, Consistency, Domain / allowed values	11 / 11 passed
Test_products.sql	intermediate.products	7	Business rule, Completeness, Consistency, Referential integrity	7 / 7 passed
Test_suppliers.sql	intermediate.suppliers	7	Completeness, Consistency, Referential integrity, Uniqueness	7 / 7 passed
Test_web_logs.sql	intermediate.web_logs	5	Business rule, Completeness, Domain / allowed values, Uniqueness	5 / 5 passed
Tests_companies.sql	intermediate.companies	12	Business rule, Completeness, Domain / allowed values, Range	12 / 12 passed
Tests_supplier_product_mapping.sql	intermediate.supplier_product_mapping	8	Business rule, Completeness, Consistency, Uniqueness	8 / 8 passed
Test_rejected_records.sql	intermediate.rejected_records	4	Completeness, Uniqueness, Domain / allowed values, Range	4 / 4 passed

Test File	Object Under Test	# Cases	Checks Covered	Result
dim_companies_tests.sql	warehouse.dim_companies	11	Business rule, Completeness, Consistency, Domain / allowed values	11 / 11 passed
dim_customers_tests.sql	warehouse.dim_customers	5	Business rule, Reconciliation, Referential integrity, Uniqueness	5 / 5 passed
dim_products_tests.sql	warehouse.dim_products	4	Reconciliation, Referential integrity, Uniqueness	4 / 4 passed
dim_supplier_product_tests.sql	warehouse.dim_supplier_product	9	Business rule, Key integrity, Range, Reconciliation	9 / 9 passed
dim_suppliers_tests.sql	warehouse.dim_suppliers	5	Business rule, Reconciliation, Referential integrity, Uniqueness	5 / 5 passed
fact_leads_tests.sql	warehouse.fact_leads	8	Business rule, Consistency, Range, Reconciliation	8 / 8 passed
fact_order_items_tests.sql	warehouse.fact_order_items	15	Business rule, Consistency, Range, Reconciliation	15 / 15 passed
fact_orders_tests.sql	warehouse.fact_orders	13	Business rule, Consistency, Domain / allowed values, Key integrity	13 / 13 passed
fact_warehouse_tests.sql	warehouse (all tables)	10	Business rule, Consistency, Reconciliation, Referential integrity	10 / 10 passed
fact_web_logs_tests.sql	warehouse.fact_web_logs	7	Business rule, Consistency, Domain / allowed values, Range	7 / 7 passed

7. Integration Test Summary

Table 7: Integration Test Summary

Test ID	Scenario	Layers	Expected Result	Result
IT-01	Source -> intermediate completeness	Source, Intermediate	Every source record (8 tables + 2 files) is in intermediate or in rejected_records	Pass - 0 differences (2,258 + 4,517 rejected CSV rows)
IT-02	Staging cleared after a successful run	Staging, Intermediate	Every staging row is in intermediate or rejected_records; then all 10 staging tables are empty	Pass - 0 staging rows left
IT-03	Intermediate -> warehouse completeness	Intermediate, Warehouse	Row counts equal for 5 dimensions (current rows) and 4 facts	Pass - 0 differences

Test ID	Scenario	Layers	Expected Result	Result
IT-04	CDC watermark correctness	Metadata, Source	Each of the 8 watermarks = MAX(updated_at) of its source table	Pass - 8 of 8 watermarks correct
IT-05	SCD Type 2 history after CDC updates	Source, Intermediate, Warehouse	Changed companies have a closed old version whose end = start of the new version; one current version each	Pass - 5 companies with history, 0 gaps/overlaps
IT-06	Point-in-time company lookup	Warehouse	Every order points to the company version valid at order time	Pass - 0 orders outside their version's validity
IT-07	Order header vs order lines	Warehouse	fact_orders.order_total = SUM(line_total) and item_count = number of lines	Pass - 0 mismatching orders
IT-08	Lead to order link (cross-source)	Intermediate, Warehouse	Converted leads carry their order_id; every order's lead_id exists in fact_leads	Pass - 0 inconsistent leads, 0 orders with unknown lead
IT-09	End-to-end incremental run	All	Latest run of all 5 stages has status 'completed'	Pass - 5 of 5 stages completed
IT-10	Idempotent re-run of warehouse loads	Warehouse	Re-running the four load_fact_*.sql scripts leaves all fact row counts unchanged	Pass - 0 rows changed by each MERGE; counts and value totals identical (orders 156,678, lines 470,263, web logs 296,703, leads 148,358)

8. KPI Validation

Table 8: KPI Validation

KPI ID	KPI	Validation Method	Expected	Actual	Result
KPI-01	Monthly Revenue Trend	Sum of monthly net revenue = non-cancelled line revenue of the same 23 complete months	4,400,274,489.23	4,400,274,489.23	Pass
KPI-02	Revenue by Company	Sum of company gross revenue = SUM(quantity x unit_price) of buyer, non-cancelled lines	5,133,378,828.46	5,133,378,828.46	Pass
KPI-03	Revenue by Product	Sum of product net revenue = total net revenue; one row per product	4,621,200,118.84	4,621,200,118.84	Pass
KPI-04	Gross Margin Analysis	Sum of gross profit = SUM(line_total - quantity x supplier_price)	954,597,910.13	954,597,910.13	Pass
KPI-05	Customer Lifetime Value	Sum of company total spend = net revenue of buyer companies	4,621,200,118.84	4,621,200,118.84	Pass
KPI-06	Customer Activity	Repeat purchase rate recomputed from fact_orders	60.31	60.31	Pass

KPI ID	KPI	Validation Method	Expected	Actual	Result
KPI-07	Geographic Sales	Sum of revenue by location = total net revenue	4,621,200,118.84	4,621,200,118.84	Pass
KPI-08	Traffic by Device	Sum of sessions = distinct (session, device) of non-bot requests	281,911	281,911	Pass
KPI-09	Geographic Web Traffic	Countries in the view = countries in non-bot web logs	3	3	Pass
KPI-10	Web Activity Quality	Sum of requests by endpoint = all web-log rows; errors = status 400-599	296703 / 59348	296703 / 59348	Pass
KPI-11	Lead Conversion Funnel	Sum of converted leads = leads with conversion_status 'Converted'	6,044	6,044	Pass
KPI-12	Lead Quality vs Order Value	Converted-lead count = independent join count; correlation within -1..1	5748 r=0.0071	5748 r=0.0071	Pass
KPI-13	Supplier Revenue	Sum of supplier revenue = total net revenue; shares add up to 100 %	4621200118.84 100%	4621200118.84 100%	Pass
KPI-14	Supplier Product Performance	Sum of supplier-product revenue = total net revenue	4,621,200,118.84	4,621,200,118.84	Pass
KPI-15	Pipeline Execution Metrics	One row per pipeline in the run log; data freshness / watermark lag populated	5 rows, 5 with freshness	5 rows, 5 with freshness	Pass

9. Defect Log

Table 9: Defect Log

Defect ID	Found In	Description	Severity	Status	Fix / Commit
D-01	Test_web_logs.sql	A leftover query at the top returned every web log	Low	Fixed	Query removed (30/09/2026)
D-02	Tests_companies.sql	A stray semicolon split one test in two	Low	Fixed	Semicolon removed
D-03	Test_categories.sql	Missing semicolon; TEST 5 checked companies instead of categories	Low	Fixed	Semicolon added; table corrected
D-04	dim_companies_tests.sql (TEST 11)	Checked the current SCD2 version instead of the first one (failed after a company change)	Medium	Fixed	Checks MIN(effective_start_date) per company

Defect ID	Found In	Description	Severity	Status	Fix / Commit
D-05	KPI 6 (vw_customer_activity)	Extra is_current filter gave 449 customers without a country	Medium	Fixed	Join on company_key only
D-06	KPI 1 (vw_monthly_revenue_trend)	Future and partial months included; 'Latest Net MoM %' card showed '--'	Medium	Fixed	Complete months only; month_num / month_start added
D-07	Data generators	Daily data was back-dated; volume ramp hid seasonality; no failed/refunded payments	High	Fixed	Date first, then customer; daily data dated today; payment by order status

10. Test Results Summary

Table 10: Test Results Summary

Test Level	Planned	Executed	Passed	Failed	Pass %
Unit	161	161	161	0	100 %
Integration	10	10	10	0	100 %
KPI Validation	15	15	15	0	100 %
Total	186	186	186	0	100 %

11. Test Evidence

Figure 1 shows that every pipeline stage of the final run completed.

Query		Query History				
1	SELECT					
2	pipeline_name, status, rows_extracted, rows_loaded, run_started_at, run_ended_at					
3	FROM metadata.pipeline_run_log					
4	ORDER BY run_started_at DESC					
5	LIMIT 10;					
6						
Data Output		Messages				
		Notifications				
		Showing rows: 1 to 10				
		Page No: 1				
	pipeline_name	status	rows_extracted	rows_loaded	run_started_at	run_ended_at
	character varying (100)	character varying (20)	integer	integer	timestamp without time zone	timestamp without time zone
1	warehouse_load	completed	[null]	1501	2026-09-30 17:32:26.594969	2026-09-30 17:33:26.293246
2	intermediate_transform	completed	[null]	297567	2026-09-30 17:31:42.741172	2026-09-30 17:32:25.471447
3	web_logs_pipeline	completed	301220	301220	2026-09-30 17:31:10.245685	2026-09-30 17:31:39.454006
4	marketing_leads_pipeli...	completed	150616	150616	2026-09-30 17:30:49.968991	2026-09-30 17:31:05.837898
5	sql_server_pipeline	completed	881	881	2026-09-30 17:30:43.129156	2026-09-30 17:30:44.729435
6	warehouse_load	completed	[null]	1498	2026-09-30 17:29:30.066425	2026-09-30 17:30:25.780681
7	intermediate_transform	completed	[null]	297165	2026-09-30 17:28:43.258518	2026-09-30 17:29:28.322206
8	web_logs_pipeline	completed	300806	300806	2026-09-30 17:28:16.144543	2026-09-30 17:28:39.986211
9	marketing_leads_pipeli...	completed	150413	150413	2026-09-30 17:27:57.621214	2026-09-30 17:28:11.602063
10	sql_server_pipeline	completed	851	851	2026-09-30 17:27:50.377578	2026-09-30 17:27:52.503623

Figure 1: Pipeline run log — all stages completed

Figure 2 shows an intermediate-layer unit test: the query returns zero rows, which means the test passed.

```

b2b_warehouse_db=# SELECT order_id, COUNT(*) AS duplicate_count FROM intermediate.orders GROUP BY order_id HAVING COUNT(*) > 1;
 order_id | duplicate_count
-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT * FROM intermediate.orders WHERE order_id IS NULL OR customer_id IS NULL OR company_id IS NULL OR order_date IS NULL OR order_status IS NULL OR payment_status IS NULL OR order_total IS NULL OR created_at IS NULL OR updated_at
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT * FROM intermediate.orders WHERE order_status NOT IN ('Pending', 'Confirmed', 'Processing', 'Shipped', 'Delivered', 'Cancelled');
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT * FROM intermediate.orders WHERE payment_status NOT IN ('pending', 'paid', 'failed', 'refunded');
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT * FROM intermediate.orders WHERE order_total < 0;
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT * FROM intermediate.orders WHERE updated_at < created_at;
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.* FROM intermediate.orders o LEFT JOIN intermediate.customers c ON c.customer_id = o.customer_id WHERE c.customer_id IS NULL;
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.order_id, o.company_id AS order_company, c.company_id AS customer_company FROM intermediate.orders o JOIN intermediate.customers c ON c.customer_id = o.customer_id WHERE o.company_id <> c.company_id;
 order_id | order_company | customer_company
-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.order_id, o.order_date, c.created_at AS customer_created_at FROM intermediate.orders o JOIN intermediate.customers c ON c.customer_id = o.customer_id WHERE o.order_date < c.created_at;
 order_id | order_date | customer_created_at
-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.* FROM intermediate.orders o LEFT JOIN intermediate.companies c ON c.company_id = o.company_id WHERE c.company_id IS NULL;
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.* FROM intermediate.orders o LEFT JOIN intermediate.marketing_leads l ON l.lead_id = o.lead_id WHERE o.lead_id IS NOT NULL AND l.lead_id IS NULL;
 order_id | customer_id | company_id | lead_id | order_date | order_status | payment_status | order_total | created_at | updated_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

```

Figure 2: Unit test query returning zero rows — intermediate

Figure 3 shows a warehouse unit test returning zero rows (passed).

```

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT company_id FROM warehouse.dim_companies WHERE is_current = TRUE GROUP BY company_id HAVING COUNT(*) > 1;
company_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT company_id FROM warehouse.dim_companies WHERE effective_end_date IS NOT NULL AND effective_end_date <= effective_start_date;
company_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.order_id FROM warehouse.fact_orders AS f LEFT JOIN warehouse.dim_customers AS c ON c.customer_key = f.customer_key WHERE c.cust
omer_key IS NULL;
order_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.order_id FROM warehouse.fact_orders AS f LEFT JOIN warehouse.dim_companies AS c ON c.company_key = f.company_key WHERE c.compan
y_key IS NULL;
order_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.order_item_id FROM warehouse.fact_order_items AS f LEFT JOIN warehouse.dim_products AS p ON p.product_key = f.product_key WHERE
p.product_key IS NULL;
order_item_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.order_item_id FROM warehouse.fact_order_items AS f LEFT JOIN warehouse.dim_suppliers AS s ON s.supplier_key = f.supplier_key WHERE s.supplier_key IS NULL;
order_item_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.log_id FROM warehouse.fact_web_logs AS f LEFT JOIN warehouse.dim_date AS d ON d.date_key = f.date_key WHERE d.date_key IS NULL;
log_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT f.lead_id FROM warehouse.fact_leads AS f LEFT JOIN warehouse.dim_date AS d ON d.date_key = f.date_key WHERE d.date_key IS NULL;
lead_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.order_id FROM warehouse.fact_orders AS o LEFT JOIN warehouse.fact_order_items AS oi ON oi.order_id = o.order_id WHERE oi.order_id IS NULL;
order_id
-----
(0 rows)

b2b_warehouse_db=#
b2b_warehouse_db=# SELECT o.order_id FROM warehouse.fact_orders AS o JOIN warehouse.fact_order_items AS oi ON oi.order_id = o.order_id GROUP BY o.order_id, o.item_count HAVING o.item_count <> COUNT(*);
order_id
-----
(0 rows)

```

Figure 3: Unit test query returning zero rows — warehouse

Figure 4 shows how history is kept for companies: a changed company has an expired version and a current version.

```
41 SELECT
42     company_id, company_key, rating, city, effective_start_date, effective_end_date, is_current
43 FROM
44     warehouse.dim_companies
45 WHERE company_id IN (
46     SELECT company_id
47     FROM warehouse.dim_companies
48     GROUP BY company_id
49     HAVING COUNT(*) > 1)
50 ORDER BY company_id, effective_start_date
51 LIMIT 12;
```

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 10

🔍

🖨️

Page No: 1

	company_id character (32)	company_key [PK] integer	rating numeric (2,1)	city character varying (100)	effective_start_date timestamp without time zone	effective_end_date timestamp without time zone	is_current boolean
1	0178cbb0c71242cdaf28bdb80253e5...	16	2.6	London	2026-02-12 05:14:12	2026-09-30 17:27:45.417458	false
2	0178cbb0c71242cdaf28bdb80253e5...	2502	2.8	Birmingham	2026-09-30 17:27:45.417458	[null]	true
3	3c916deb199f40baabdfcae0b56109...	606	3.1	New York	2025-08-01 03:57:10	2026-09-30 17:27:45.392565	false
4	3c916deb199f40baabdfcae0b56109...	2503	3.1	New York	2026-09-30 17:27:45.392565	[null]	true
5	88245c3e81444b94b9e3c21043c88f...	1373	3.8	Austin	2026-04-18 01:54:49	2026-09-30 16:22:08.695862	false
6	88245c3e81444b94b9e3c21043c88f...	2501	3.7	Seattle	2026-09-30 16:22:08.695862	[null]	true
7	8b4c4251f551408b8ed1a4e0ca2a3f...	1410	4.3	New York	2026-09-08 05:33:19	2026-09-30 17:30:38.062934	false
8	8b4c4251f551408b8ed1a4e0ca2a3f...	2505	4.4	Chicago	2026-09-30 17:30:38.062934	[null]	true
9	b71a61ae2a4249eebf73bb0801429c...	1836	3.4	Seattle	2024-12-12 14:13:54	2026-09-30 17:27:45.430529	false
10	b71a61ae2a4249eebf73bb0801429c...	2504	3.3	New York	2026-09-30 17:27:45.430529	[null]	true

Figure 4: SCD Type 2 — companies with more than one version

Figure 5 shows the idempotency test: re-running the warehouse loads leaves every row count unchanged.

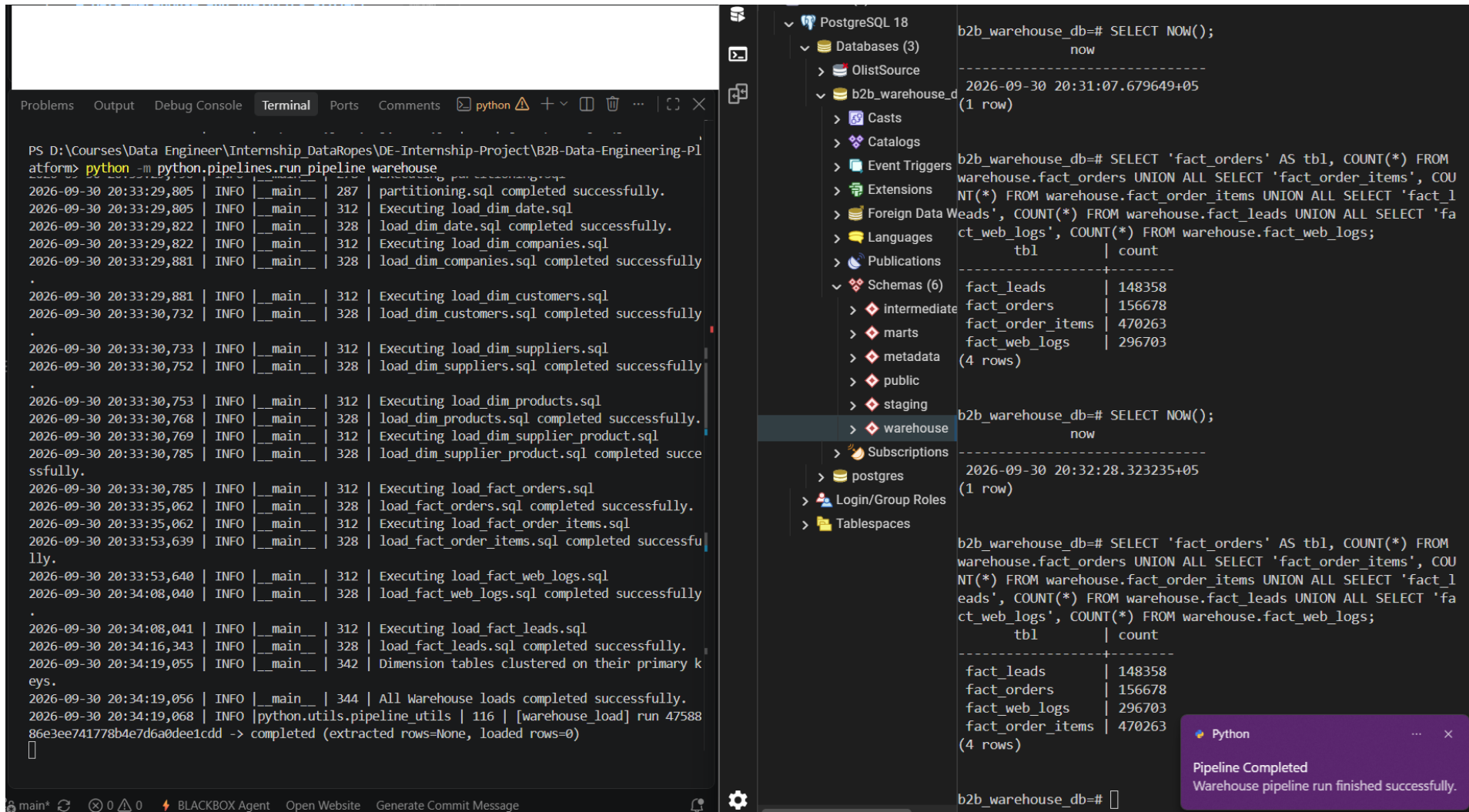


Figure 5: Idempotency re-run — row counts unchanged

Figure 6 shows the layer reconciliation: every CSV row is either loaded or rejected, none is lost.

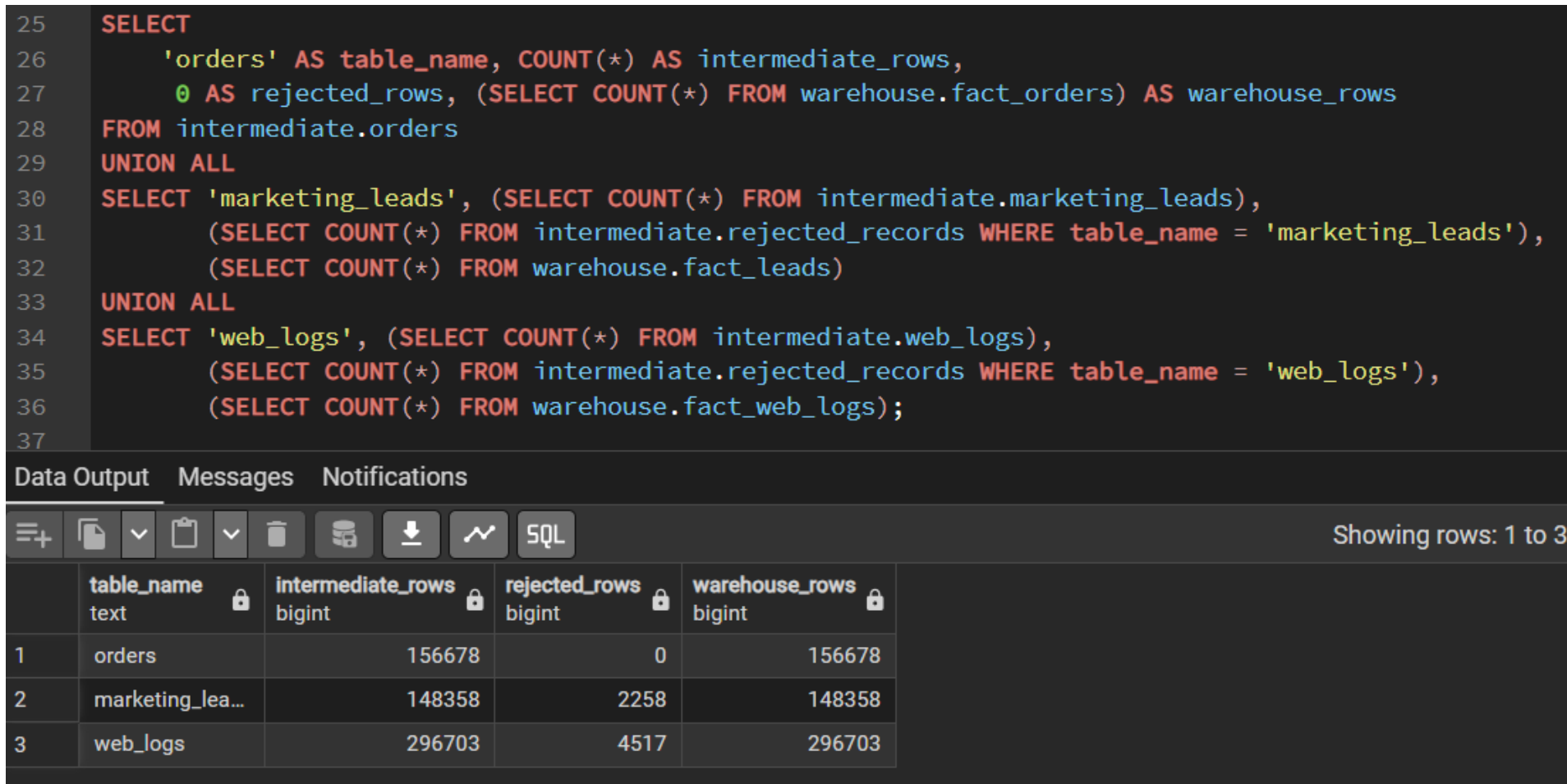


Figure 6: Layer reconciliation — every CSV row is loaded or rejected

Figure 7 shows the rejected rows per table and reason, as stored in `intermediate.rejected_records`.

```

76 SELECT table_name, reject_reason, COUNT(*) AS rejected_rows
77 FROM intermediate.rejected_records
78 GROUP BY table_name, reject_reason
79 ORDER BY table_name, reject_reason;
80

```

Data Output Messages Notifications			
	table_name character varying (50)	reject_reason character varying (100)	rejected_rows bigint
1	marketing_leads	invalid estimated_order_value	741
2	marketing_leads	invalid lead_score	723
3	marketing_leads	missing required field	794
4	web_logs	invalid bytes_sent	1106
5	web_logs	invalid http_method	1133
6	web_logs	invalid status_code	1119
7	web_logs	missing required field	1159

Figure 7: Rejected rows by table and reason

Figure 8 shows the integration test summary: failed_rows is 0 for every test.

```

264  -- =====
265  -- SUMMARY: all automatic tests above in one result
266  -- one row per test, failed_rows must be 0 for every test
267  -- =====
268
269  SELECT 'IT-02 staging cleared' AS test_name,
270        (SELECT COUNT(*) FROM staging.orders) + (SELECT COUNT(*) FROM staging.order_items)
271        + (SELECT COUNT(*) FROM staging.marketing_leads) + (SELECT COUNT(*) FROM staging.web_logs)
272        + (SELECT COUNT(*) FROM staging.companies) + (SELECT COUNT(*) FROM staging.customers)
273        + (SELECT COUNT(*) FROM staging.suppliers) + (SELECT COUNT(*) FROM staging.products)
274        + (SELECT COUNT(*) FROM staging.categories) + (SELECT COUNT(*) FROM staging.supplier_product_mapping) AS failed_rows
275
276  UNION ALL
277  SELECT 'IT-03 orders: intermediate = warehouse',
278        ABS((SELECT COUNT(*) FROM intermediate.orders) - (SELECT COUNT(*) FROM warehouse.fact_orders))
279
280  UNION ALL
281  SELECT 'IT-03 order_items: intermediate = warehouse',
282        ABS((SELECT COUNT(*) FROM intermediate.order_items) - (SELECT COUNT(*) FROM warehouse.fact_order_items))
283
284  UNION ALL
285  SELECT 'IT-03 leads: intermediate = warehouse',
286        ABS((SELECT COUNT(*) FROM intermediate.marketing_leads) - (SELECT COUNT(*) FROM warehouse.fact_leads))
287
288  UNION ALL
289  SELECT 'IT-03 web_logs: intermediate = warehouse',
290        ABS((SELECT COUNT(*) FROM intermediate.web_logs) - (SELECT COUNT(*) FROM warehouse.fact_web_logs))
291

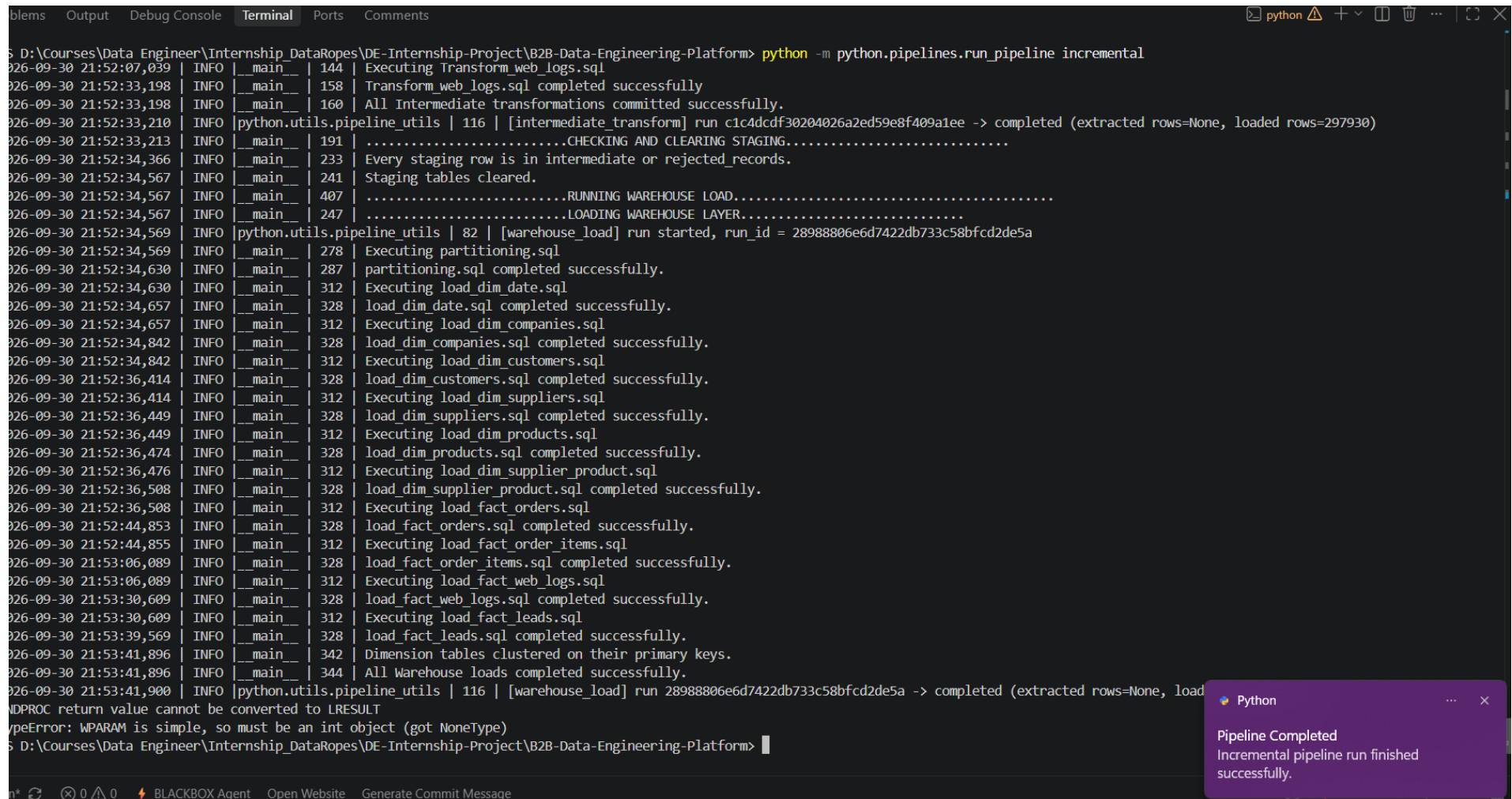
```

Data Output Messages Notifications

	test_name text	failed_rows bigint
1	IT-02 staging cleared	0
2	IT-03 orders: intermediate = warehouse	0
3	IT-03 order_items: intermediate = warehouse	0
4	IT-03 leads: intermediate = warehouse	0
5	IT-03 web_logs: intermediate = warehouse	0
6	IT-03 companies: intermediate = current dim rows	0
7	IT-04 watermarks filled (8 expected)	0
8	IT-05 companies without exactly 1 current version	0
9	IT-06 orders linked to wrong company version	0
10	IT-07 order total <=> sum of lines	0
11	IT-08 converted lead without order (or opposite)	0
12	IT-09 latest pipeline runs not completed	0

Figure 8: Integration test summary — failed_rows is 0 for every test

Figure 9 shows the terminal output of an incremental run: every stage completes, staging is checked and cleared and the warehouse is loaded.



```

D:\Courses\Data Engineer\Internship_DataRopes\DE-Internship-Project\B2B-Data-Engineering-Platform> python -m python.pipelines.run_pipeline incremental
026-09-30 21:52:07,039 | INFO | __main__ | 144 | Executing Transform_web_logs.sql
026-09-30 21:52:33,198 | INFO | __main__ | 158 | Transform_web_logs.sql completed successfully
026-09-30 21:52:33,198 | INFO | __main__ | 160 | All Intermediate transformations committed successfully.
026-09-30 21:52:33,210 | INFO | python.utils.pipeline_utils | 116 | [intermediate_transform] run c1c4dcdf30204026a2ed59e8f409a1ee -> completed (extracted rows=None, loaded rows=297930)
026-09-30 21:52:33,213 | INFO | __main__ | 191 | .....CHECKING AND CLEARING STAGING.....
026-09-30 21:52:34,366 | INFO | __main__ | 233 | Every staging row is in intermediate or rejected_records.
026-09-30 21:52:34,567 | INFO | __main__ | 241 | Staging tables cleared.
026-09-30 21:52:34,567 | INFO | __main__ | 407 | .....RUNNING WAREHOUSE LOAD.....
026-09-30 21:52:34,567 | INFO | __main__ | 247 | .....LOADING WAREHOUSE LAYER.....
026-09-30 21:52:34,569 | INFO | python.utils.pipeline_utils | 82 | [warehouse_load] run started, run_id = 28988806e6d7422db733c58bfcd2de5a
026-09-30 21:52:34,569 | INFO | __main__ | 278 | Executing partitioning.sql
026-09-30 21:52:34,630 | INFO | __main__ | 287 | partitioning.sql completed successfully.
026-09-30 21:52:34,630 | INFO | __main__ | 312 | Executing load_dim_date.sql
026-09-30 21:52:34,657 | INFO | __main__ | 328 | load_dim_date.sql completed successfully.
026-09-30 21:52:34,657 | INFO | __main__ | 312 | Executing load_dim_companies.sql
026-09-30 21:52:34,842 | INFO | __main__ | 328 | load_dim_companies.sql completed successfully.
026-09-30 21:52:34,842 | INFO | __main__ | 312 | Executing load_dim_customers.sql
026-09-30 21:52:36,414 | INFO | __main__ | 328 | load_dim_customers.sql completed successfully.
026-09-30 21:52:36,414 | INFO | __main__ | 312 | Executing load_dim_suppliers.sql
026-09-30 21:52:36,449 | INFO | __main__ | 328 | load_dim_suppliers.sql completed successfully.
026-09-30 21:52:36,449 | INFO | __main__ | 312 | Executing load_dim_products.sql
026-09-30 21:52:36,474 | INFO | __main__ | 328 | load_dim_products.sql completed successfully.
026-09-30 21:52:36,476 | INFO | __main__ | 312 | Executing load_dim_supplier_product.sql
026-09-30 21:52:36,508 | INFO | __main__ | 328 | load_dim_supplier_product.sql completed successfully.
026-09-30 21:52:36,508 | INFO | __main__ | 312 | Executing load_fact_orders.sql
026-09-30 21:52:44,853 | INFO | __main__ | 328 | load_fact_orders.sql completed successfully.
026-09-30 21:52:44,855 | INFO | __main__ | 312 | Executing load_fact_order_items.sql
026-09-30 21:53:06,089 | INFO | __main__ | 328 | load_fact_order_items.sql completed successfully.
026-09-30 21:53:06,089 | INFO | __main__ | 312 | Executing load_fact_web_logs.sql
026-09-30 21:53:30,609 | INFO | __main__ | 328 | load_fact_web_logs.sql completed successfully.
026-09-30 21:53:30,609 | INFO | __main__ | 312 | Executing load_fact_leads.sql
026-09-30 21:53:39,569 | INFO | __main__ | 328 | load_fact_leads.sql completed successfully.
026-09-30 21:53:41,896 | INFO | __main__ | 342 | Dimension tables clustered on their primary keys.
026-09-30 21:53:41,896 | INFO | __main__ | 344 | All Warehouse loads completed successfully.
026-09-30 21:53:41,900 | INFO | python.utils.pipeline_utils | 116 | [warehouse_load] run 28988806e6d7422db733c58bfcd2de5a -> completed (extracted rows=None, load
NDPROC return value cannot be converted to LRESULT
ypeError: WPARAM is simple, so must be an int object (got NoneType)
D:\Courses\Data Engineer\Internship_DataRopes\DE-Internship-Project\B2B-Data-Engineering-Platform>

```

Python

Pipeline Completed
Incremental pipeline run finished successfully.

Figure 9: Terminal after an incremental run — staging checked and cleared

12. Sign-off

Table 11: Sign-off

Role	Name	Decision	Signature	Date
Tester	Syed Kazim Raza	All tests passed - accepted		30/09/2026
Team Lead				